HW4 — Interactive theorem proving in Lean 4

CS 3892 / CS 5892 · Formal Methods: Safe and Trustworthy AI · Fall 2026

Out	Tuesday, November 3
Due	**Thursday, November 12, 11:59 p.m. US Central**
Weight	4% of the course grade (100 points)
Tools	Lean 4, v4.33.0, core library only (no Mathlib)
Zero-install path	the HW4 notebook in Google Colab; the Lean web editor for trying proofs
Submit	Brightspace → Assignments → *HW4 — Interactive theorem proving (Lean 4)*

Start here: github.com/ttj/cs3892-examples has the starters in `homework/hw4-lean/`, and the HW4 notebook installs Lean and runs the checker in Colab. Every theorem you have to prove is there with `sorry` in place of its proof.

What you need, and when you get it. Tuesday Nov 3 (terms, types, tactics, `sorry`, Curry–Howard) covers **Parts 1 and 2**. Thursday Nov 5 (transition systems in Lean: `Reachable`, the init and step lemmas, strengthening, a parameter `N`) covers **Parts 3 and 4**. Keep the Part 5 log as you go.

What this is for

A model checker answers a question about one finite model. A proof assistant checks an argument, and an argument can cover infinitely many models at once: every capacity `N`, not just `N = 10`. The price is that you have to supply the argument, and the hard part of the argument is the same one HW3 Part 2 asked for: an invariant that is **inductive**, not just true. Lean also makes a precise judge of an AI assistant's proofs: the compiler either accepts a proof or names exactly what is wrong with it.

Running the tools

In Colab. Open the HW4 notebook and run it top to bottom: it installs Lean v4.33.0 (a minute or two, once per runtime) and prints the checker's table. Edit the `Part…lean` files with the file browser (folder icon, left), save, and re-run the last cell. Colab has no Lean editor, so you will not see goals as you type; for that, use VS Code with the Lean 4 extension, or the Lean web editor. **A Colab runtime is deleted when it disconnects: download your files before you leave.**

On your machine, or in a Codespace:

```
git clone --depth 1 https://github.com/ttj/cs3892-examples.git
cd cs3892-examples/homework/hw4-lean
python3 check_hw4.py
```

`check_hw4.py` installs elan (the Lean version manager, no root needed) and Lean v4.33.0 the first time, if it finds no `lake`. While you work on one part, `lake build HW4.Part3Bucket` is faster.

What counts as proved. For every required theorem the checker reports *proved*, *uses sorry*, *disallowed axiom*, *statement changed*, or *part does not build*. **Proved** means: the part builds, the statement is exactly the one you were given, and `#print axioms` lists nothing beyond `propext`, `Classical.choice` and `Quot.sound`. A `sorry` compiles and proves nothing; it shows up as `sorryAx`. `native_decide` and any `axiom` you declare yourself are rejected the same way. `HW4/Defs.lean` holds every given definition, and the checker refuses a changed copy.

Tactics you need

Tactic	Does
`rfl`, `decide`	close a goal by computation
`omega`	linear arithmetic over `Nat` and `Int`
`intro h`	assume the left side of `→` (or take a `∀` variable)
`exact e`	close the goal with a term `e` of exactly that type
`apply f`	work backwards from `f`'s conclusion
`constructor`	split a goal `A ∧ B` (or `A ↔ B`) into its parts
`left`, `right`	choose a side of `A ∨ B`
`obtain ⟨x, hx⟩ := h`, `cases h with …`	take apart a hypothesis: `∃`, `∧`, `∨`, or a step
`induction n with …`	induction on a natural number (or on how a state was reached)
`by_cases h : p`	split on whether `p` holds (classical)
`simp only [lemmas] at h`	rewrite with the given lemmas, in the goal or in `h`
`rw [e]`, `show t`	rewrite with an equation; restate the goal in a definitionally equal form

---

Part 1 — A tactic ladder *(15 points)*

`HW4/Part1Ladder.lean`: seven rungs, each practising one idea: computation, linear arithmetic, a conjunction, an existential, an implication (Curry–Howard: a proof of `P → R` is a function), induction on a recursive definition, and case analysis on a finite type. If you have not used Lean before, play the first worlds of the Natural Number Game first.

Part 2 — Discrete maths, machine-checked *(20 points)*

`HW4/Part2Sets.lean`: five set identities, with sets as predicates as in week 4. Antisymmetry of `⊆`, De Morgan for an intersection, `∪` distributing over `∩`, a difference with a union, and `C ⊆ A ∩ B ↔ C ⊆ A ∧ C ⊆ B`. In your report, say **which one needs classical logic** (`by_cases`, the law of excluded middle) and why the others do not.

Part 3 — A property that is true, and not inductive *(30 points)*

`bucket10` is a rate limiter for an agent's actions: 10 tokens; each action spends one; an empty bucket is refilled. The property `P b := b.used ≤ 10` (at most 10 actions per window) holds in every reachable state.

1. *(6)* Prove `P_not_inductive`: give a state that satisfies `P` and a step out of it that breaks `P`. In your report: is that state reachable, and why does the answer not rescue `P`?
2. *(6)* Prove `Q_not_inductive`: the obvious strengthening, "`used ≤ 10` and `left ≤ 10`", fails too. Say in one sentence what it still fails to rule out.
3. *(12)* **Define `Inv`** and prove `inv_init` and `inv_step`. Choosing `Inv` is the whole assignment. In your report, say what `Inv` expresses about the system and why every step preserves it.
4. *(6)* Prove `used_le_10` from them, using `inductive_invariant_holds` (in `Defs.lean`).

Part 4 — Every capacity at once *(20 points)*

`bucket N` has `N` tokens.

1. *(10)* Prove `used_le_N`: the bound holds for **every** `N`.
2. *(6)* Prove the helper `reach_k`, by induction on `k`: from a full bucket, after `k` actions, the state is `⟨k, N − k⟩`.
3. *(2)* Prove `bound_tight`: some reachable state has `used = N`, so the bound cannot be improved.
4. *(2, written)* HW2's model checker proved properties of a counter with a fixed cap. Why could it not prove `used_le_N`? What would it give you for `N = 10`, and what for "all `N`"?

Part 5 — AI component: a proof-partner log *(15 points)*

Use an AI assistant as a proof partner on at least four of the theorems, including at least one from Part 3. Keep `ai_log.md`: for each theorem, who wrote the final proof, how many iterations it took before Lean accepted it, and **every lemma or tactic name the assistant invented**, with Lean's error message verbatim. Then answer the three questions at the end of the log. Lean is the ground truth here: a proof is right when it compiles and the checker says *proved*, whatever the assistant claims.

What to submit

One archive or repository link containing:

1. Your four `HW4/Part…lean` files. `python3 check_hw4.py --strict` is the acceptance test: it exits 0 only when every theorem is proved. Points are per theorem, as the checker's table shows.
2. A short report, Markdown or PDF, about one page. It answers the questions in Parts 2, 3 and 4.
3. `ai_log.md` (Part 5), and the AI-use disclosure (below).

Late policy

Four late days for the semester, in whole-day units, no more than two on any one homework, no questions asked. Beyond that, 20% per day.

Collaboration and AI use

Discussion is encouraged; submitted work must be your own. **AI assistants are allowed and expected**, and Part 5 is about using one. Two conditions:

1. **Disclose.** Say what you used and for what.
2. **Verify.** You are responsible for everything you submit. Here verification is mechanical: the checker either reports *proved* or it does not.